Module II : File Systems and Device Management

File Systems: Overview, File Organization and Access, File Directories, File Sharing, Record Blocking, **Secondary Storage Management**, File System Security, UNIX File System, Linux Virtual File System, Windows File System, Case studies of advanced file systems (ext4, ZFS, Btrfs). **Device Management**: I/O Devices, Organization of the I/O Function, Operating System Design Issues, I/O Buffering, I/O hub, **Disk Scheduling Policies**: FIFO, PRIORITY, SSTF, SCAN, C-SCAN, Linux I/O, Windows I/O.

Assignment 4

Simulation of Copy Command

https://archive.org/details/DesignUNIXOperatingSystem/page/8/mode/2up

- Write a program to demonstrate Linux file system internals by using file and directory related system call to create, access, modify, and manage files and directories along with appropriate error handling.

- OVERVIEW:
- Files and File Systems:
- From the user's point of view, one of the most important parts of an operating system is the file system.
- The file system provides the abstractions associated with secondary storage.
- The file system permits users to create data collections, called files, with desirable properties, such as:
- Long-term existence: Files are stored on disk or other secondary storage and do not disappear when a user logs off.
- Sharable between processes: Files have names and can have associated access permissions that permit controlled sharing.
- Structure: Depending on the file system, a file can have an internal structure that is convenient for particular applications.
- Files can be organized into hierarchical or more complex structure to reflect the relationships among files

- Any file system provides not only a means to store data organized as files, but a collection of functions that can be performed on files.

Operations include the following:

- Create: A new file is defined and positioned within the structure of files.
- Delete: A file is removed from the file structure and destroyed.
- Open: An existing file is declared to be “opened” by a process, allowing the process to perform functions on the file.
- Close: The file is closed with respect to a process, so that the process no longer may perform functions on the file, until the process opens the file again.
- Read: A process reads all or a portion of the data in a file.
- Write: A process updates a file, either by adding new data that expands the size of the file or by changing the values of existing data items in the file.

- On UNIX and UNIX-like systems, the basic file structure is just a stream of bytes. For example, a C program is stored as a file but does not have physical fields, records, and so on.

- File Management Systems:
- A file management system is that set of system software that provides services to users and applications in the use of files.
- The only way that a user or application may access files is through the file management system.
- Following are the objectives for a file management system:
  - To meet the data management needs and requirements of the user, which include storage of data and the ability to perform the aforementioned operations
- To guarantee, to the extent possible, that the data in the file are valid
- To optimize performance, both from the system point of view in terms of overall throughput and from the user's point of view in terms of response time
- To provide I/O support for a variety of storage device types.
- To minimize or eliminate the potential for lost or destroyed data.
- To provide a standardized set of I/O interface routines to user processes.
- To provide I/O support for multiple users, in the case of multiple-user systems.

- System calls for working with files and directories in Linux:
- File descriptors:
- The operating system assigns internally to each opened file a descriptor or an identifier (this is a positive integer).
- When opening or creating a new file the system returns a file descriptor to the process that executed the call.
- Each application has its own file descriptors.
- By convention, the first three file descriptors are opened at the beginning of each process. The 0 file descriptor identifies the standard input, 1 identifies the standard output and 2 the standard output for errors.
- The rest of the descriptors are used by the processes when opening an ordinary, pipe or special file, or directories. There are five system calls that generate file descriptors: create, open, fcntl, dup and pipe.

- O_RDONLY
- Opens the file for reading.
- O_WRONLY
- Opens the file for writing.
- O_RDWR
- The file is opened for reading and writing.
- O_APPEND
- It writes successively to the end of the file.
- O_CREAT
- The file is created in case it didnot already exist.
- System call READ

- System call READ
- When we want to read a certain number of bytes starting from the current position in a file, we use the read call. The syntax is:
- ssize_t read(int fd, void* buf, size_t noct);
- The function returns the number of bytes read, 0 for end of file (EOF) and -1 in case an error occurred.
- System call WRITE
- For writing a certain number of bytes into a file starting from the current position we use the write call. Its syntax is:
- ssize_t write(int fd, const void* buf, size_t noct);
- The function returns the number of bytes written and the value -1 in case of an error.

- System call CLOSE
- For closing a file and thus eliminating the assigned descriptor we use the system call close.
- int close(int fd);
- The function returns 0 in case of success and -1 in case of an error. At the termination of a process an open file is closed anyway.
-

- **Permission Groups**
- Each file and directory has three user based permission groups:

- owner – The Owner permissions apply only to the owner of the file or directory, they will not impact the actions of other users.
- group – The Group permissions apply only to the group that has been assigned to the file or directory, they will not affect the actions of other users.
- all users – The All Users permissions apply to all other users on the system, this is the permission group that you want to watch the most.

- Permission Types:
- Each file or directory has three basic permission types:

- read – The Read permission refers to a user's capability to read the contents of the file.
- write – The Write permissions refer to a user's capability to write or modify a file or directory.
- execute – The Execute permission affects a user's capability to execute a file or view the contents of a directory.

- “ls -l”
- Modifying the Permissions the command ***chmod***
- Using Binary References to Set permissions
- A sample permission string would be **chmod 640 file1**, which means that the owner has read and write permissions, the group has read permissions, and all other user have no rights to the file.
- The numbers are a binary representation of the rwx string.
- ***r*** = *4*
- ***w*** = *2*
- ***x*** = *1*

- argc (Argument Count)
- argc is an integer that holds the total number of command-line arguments, including the name of the program itself.
- Its value is always at least 1 because argv[0] is always the program's name.
- Use argc primarily to check if the user has provided the required number of arguments, ensuring correct program usage.
- argv (Argument Vector)
- argv is an array of strings (more precisely, an array of character pointers, char *argv[] or char **argv).
- Each element in the array points to a null-terminated string representing an individual argument.
- argv[0] contains the name or path of the executed program (e.g., ./myprogram).

```
#include <fcntl.h>
char buffer[2048];
int version = 1;        /* Chapter 2 explains this */

main(argc, argv)
        int argc;
        char *argv[];
{
        int fdold, fdnew;

        if (argc != 3)
        {
                printf("need 2 arguments for copy program\n");
                exit(1);
        }
        fdold = open(argv[1], O_RDONLY);    /* open source file read only */
        if (fdold == -1)
        {
                printf("cannot open file %s\n", argv[1]);
                exit(1);
        }
        fdnew = creat(argv[2], 0666);    /* create target file rw for all */
        if (fdnew == -1)
        {
                printf("cannot create file %s\n", argv[2]);
                exit(1);
        }
        copy(fdold, fdnew);
        exit(0);
}

copy(old, new)
        int old, new;
{
        int count;

        while ((count = read(old, buffer, sizeof(buffer))) > 0)
                write(new, buffer, count);
}
```

Figure 1.3. Program to Copy a File

The program copies any files supplied to it as arguments, provided it has permission to *open* the existing file and permission to create the new file. The file can be a file of printable characters, such as the source code for the program, or it can contain unprintable characters, even the program itself. Thus, the two

- copy oldfile newfile
- oldfile is the name of the existing file
- newfile is the name of the ‘new file.
- The system invokes main, supplying argc as the number of parameters. in the list argv, and initializing each member of the array argv to point to a user-supplied parameter.
- In the example above, argc is 3, argv[0] points to the character string copy (the program name is conventionally the Oth parameter), argv[1] points to the character string oldfile, and argv[2] points to the character string newfile.
- The program then checks that it has been invoked with the proper number of parameters. If so, it invokes the. open system call “‘read-only” for the file oldfile, and if the system call succeeds, invokes the create system call to create newfile. The permission modes on the newly created file

- System call OPEN:
- #include <sys/types.h>

- #include <sys/stat.h>

- #include <fcntl.h>

- int open(const char *path, int flags,... /* mode_t mod */);

- This function returns the file descriptor or in case of an error -1

- https://www.linuxfoundation.org/blog/blog/classic-sysadmin-understanding-linux-file-permissions

Linux System Calls

- Linux system calls serve as the interface between user-level applications and the operating system kernel, allowing programs to request privileged services like file access, process management, and network communication.
- User-space applications run in an unprivileged mode and cannot directly access hardware or kernel memory. System calls enforce a controlled and secure way to access these critical resources by switching the CPU to a privileged kernel mode.
- Programmers typically use standard library functions (like printf or fopen) in C, which act as wrapper functions that handle the low-level details of invoking the correct system call.

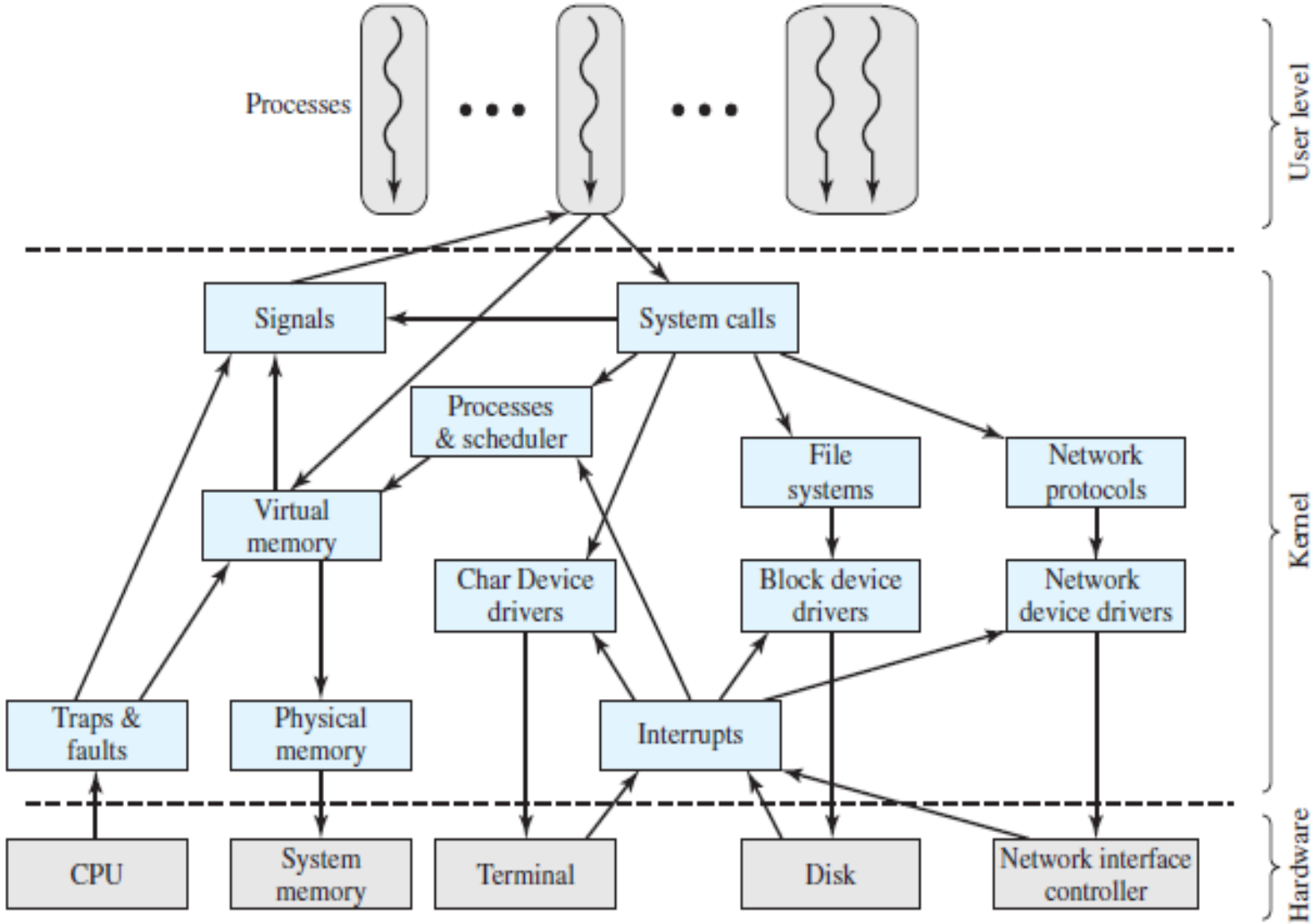
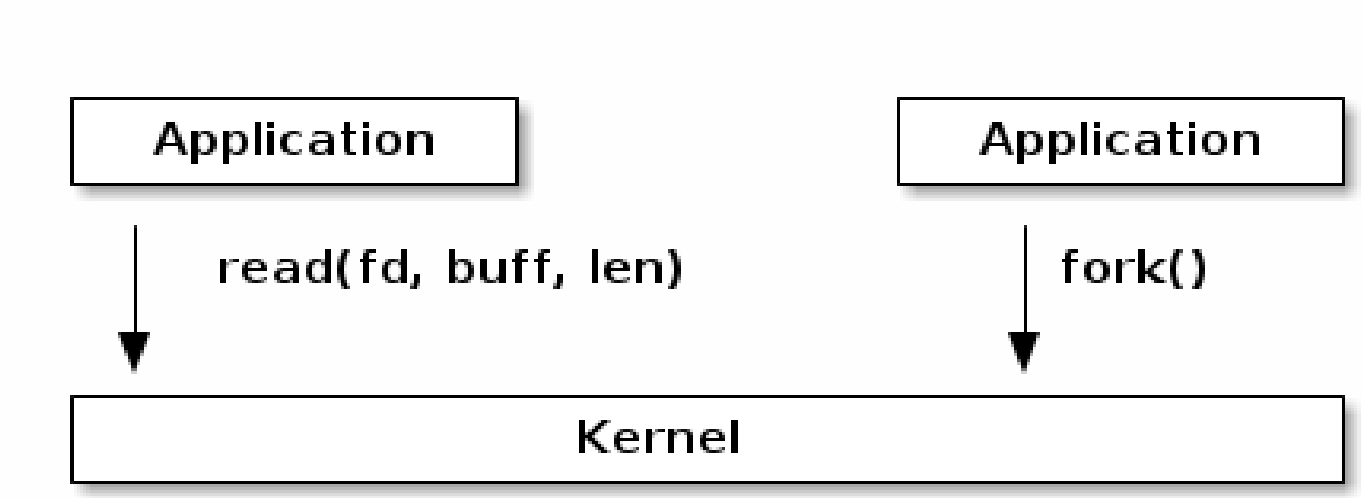


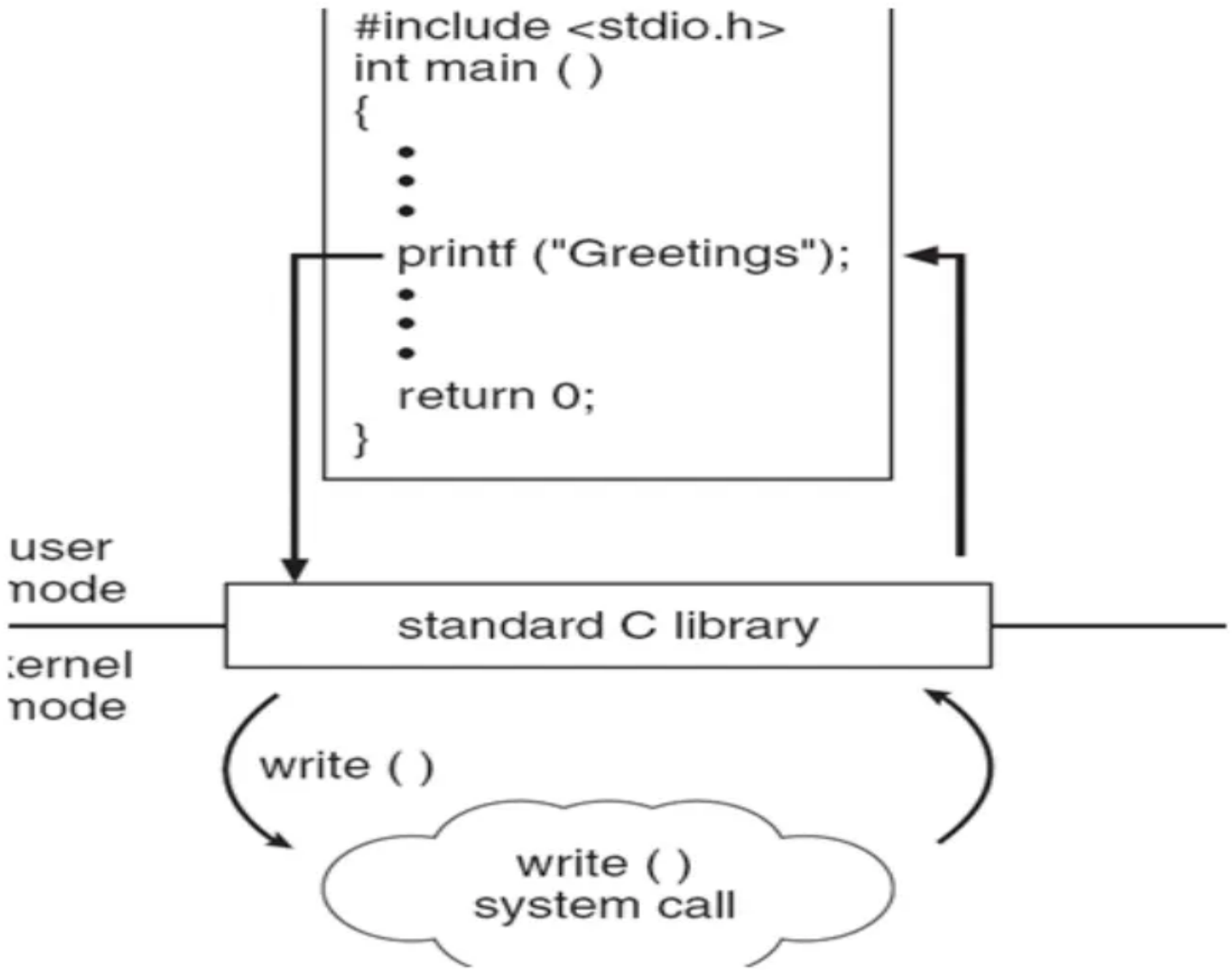
Figure 2.18 **Linux Kernel Components**

Linux system calls implementation

- At a high level system calls are "services" offered by the kernel to user applications and they resemble library APIs in that they are described as a function call with a name, parameters, and return value.



- System Calls do the following:
- setup information to identify the system call and its parameters
- trigger a kernel mode switch
- retrieve the result of the system call
- Why Are System Calls Important?
- User applications cannot access hardware or kernel space directly due to security and stability reasons.
- System calls act as controlled entry points to access these features safely.


#include <stdio.h>
int main ( )
{
printf ("Greetings");
return 0;
}
user
node
standard C library
ernel
node
write ( )
write ( )
system call